

RESEARCH

Open Access



Cross: a cloud-native approach to automated remediation and self-healing in cyber-physical systems

Obinna Johnphill¹, Ali Safaa Sadiq^{1*} , Omprakash Kaiwartya¹ and Mohammed Adam Taheir²

Abstract

Cyber-Physical Systems (CPS) operate in increasingly complex and security-critical environments where system faults, misconfigurations, and cyberattacks can compromise safety, availability, and operational integrity. This paper presents CROSS (Cross-platform Remediation and Observability Self-Healing System), a cloud-native, cross-platform approach that extends the self-healing paradigm beyond anomaly detection to encompass autonomous, security-aware remediation. Building upon the Log Intelligence and Self-Healing System (LISH) (Johnphill et al. 2023a), which utilised CountVectorizer and Multinomial Naive Bayes (MNB) for log-based anomaly classification, CROSS introduces a policy-driven remediation layer that executes context-specific recovery actions such as service restarts, system updates, device reboots, and configuration enforcement across Android, Linux, macOS, and Windows. Prometheus-based observability (Pai and Srinivas 2024) provides fine-grained telemetry on anomalies and remedial actions, enabling continuous monitoring, auditability, and adaptive security governance. Experimental evaluation demonstrates measurable reductions in mean time to recovery (MTTR) and improvements in anomaly containment and resilience across heterogeneous CPS environments.

Although CROSS includes mechanisms that are applicable to cybersecurity scenarios, the present evaluation focuses on operational anomalies rather than explicit attack-induced behaviours. Accordingly, its cybersecurity relevance is framed as an architectural capability, with empirical security benchmarking identified as future work. The proposed approach bridges the gap between anomaly detection and active cyber defence, embedding explainable, automated remediation within the operational lifecycle of CPS.

Keywords Cyber-physical systems, Self-healing, Remediation, Anomaly detection, System resilience, Observability, Cloud-native systems

Introduction

Cyber-Physical Systems (CPS) underpin critical national infrastructure across sectors such as healthcare, transportation, energy, and manufacturing. As these systems become more interconnected and software-dependent, they inherit a growing range of vulnerabilities that expose them to both accidental faults and deliberate cyberattacks (Johnphill et al. 2023a; Loh and Thing 2023). The convergence of operational technology (OT) and information technology (IT) has expanded the attack surface, allowing misconfigurations or malware to cascade across

*Correspondence:

Ali Safaa Sadiq
ali.sadiq@ntu.ac.uk

¹ Cyber Security Research Group (CSRG), Department of Computer Science, Nottingham Trent University, Clifton Lane, Nottingham NG11 8NS, UK

² Faculty of Technology Sciences, Zalingei University, Zalingei, Sudan

interdependent subsystems, compromising safety and availability.

Traditional fault-tolerance mechanisms restore functionality after benign faults but are often inadequate against security-induced anomalies. Hence, the ability to autonomously detect, diagnose, and respond to incidents is essential for maintaining resilient CPS. Advances in artificial intelligence (AI) and machine learning (ML) now enable adaptive fault management capable of operating with minimal human intervention (Cheng et al. 2023). These techniques allow CPS to identify deviations from normal behaviour, classify anomalies, and initiate corrective actions that preserve operational continuity even under hostile conditions.

Self-healing systems have emerged as a promising paradigm for autonomously mitigating both technical faults and security breaches. A self-healing CPS continuously monitors operational telemetry, detects anomalies, and performs automated recovery to maintain service quality. However, many existing approaches employ static, pre-defined responses that are insufficient for dynamic, multi-platform environments (Vervaeke 2021; Naqvi et al. 2022). This rigidity limits responsiveness during unexpected faults or cyber incidents, highlighting the need for flexible, intelligent remediation strategies that operate seamlessly across diverse operating contexts.

Recent developments in cloud computing, edge intelligence, and container orchestration now make it feasible to execute remediation processes at scale. Cloud platforms enable rapid deployment and telemetry analysis, while edge computing reduces latency by enabling localised decision-making for time-sensitive applications such as autonomous vehicles and medical devices. By combining these paradigms, self-healing CPS can detect anomalies in near real time and respond promptly through distributed remediation workflows. This hybrid approach not only improves recovery speed but also enhances resilience by ensuring isolated compromises do not escalate into systemic failures.

Building on this foundation, we introduce CROSS, the Cross-platform Remediation and Observability Self-Healing System. CROSS extends our previous Log Intelligence and Self-Healing (LISH) framework, which employed CountVectorizer and Multinomial Naïve Bayes (MNB) to classify anomalies from multi-platform system logs. While LISH focused on fault detection, CROSS operationalises the post-detection phase by autonomously executing platform-specific remedial actions across Android, Linux, macOS, and Windows environments. In addition, CROSS integrates Prometheus-based observability to expose real-time telemetry of detected anomalies, remediation events, and system recovery outcomes. This facilitates auditability and aligns

the framework with modern DevSecOps monitoring standards.

Although this study uses general-purpose operating systems as representative CPS platforms, the architecture and methodology are transferable to industrial CPS contexts. The operating-system layer in controllers, embedded devices, and production networks exhibits comparable behavioural patterns and failure signatures, making OS-level anomaly classification a scalable proxy for CPS-level diagnostics. CROSS therefore bridges conventional IT observability with OT resilience, offering a unified pathway toward autonomous, cross-domain self-healing.

This research addresses two key questions: (1) Can a cross-platform self-healing system autonomously detect and remediate both operational and security-related faults in real time using only preprocessed log data? (2) How effectively can remediation be executed across heterogeneous environments under constraints such as missing tooling, sandboxed processes, or limited connectivity?

From these, two hypotheses are proposed: (H1) A log-driven self-healing system can autonomously execute OS-specific corrective actions with minimal human oversight while maintaining secure operational continuity. (H2) Platform-specific limitations (for example, unavailable `systemd`, PowerShell, or ADB) can be mitigated through simulation and fallback mechanisms to sustain uninterrupted remediation.

This paper makes the following contributions:

- *CROSS Framework*: A cloud-native, cross-platform self-healing system integrating log-based anomaly classification with automated, policy-driven remediation.
- *Cyber-Resilience Orientation*: Embeds security awareness and autonomous defence within the remediation workflow.
- *Integrated Observability*: Provides fine-grained monitoring and transparency through Prometheus metrics and Grafana (Siddiqui et al. 2023) dashboards.
- *Adaptive Fallback Logic*: Introduces simulated remediation routines for constrained or tool-limited environments.
- *Scientific Evaluation*: Demonstrates efficiency across four operating systems using metrics such as precision, recall, MTTR, and resource impact.

The remainder of this paper is structured as follows: Sect. 2 reviews recent work on self-healing and resilience in distributed CPS; Sect. 3 presents the CROSS architecture and operational workflow; Sect. 5

describes implementation and evaluation; Sect. 6 presents simulation results; Sect. 7 discusses implications; Sect. 8 concludes and outlines future directions.

Related work

Self-healing and resilience strategies have long been explored across critical domains such as power distribution, cloud computing, and CPS. Early approaches predominantly addressed safety and fault tolerance rather than cybersecurity, focusing on restoring service availability after physical or software faults. However, with the increasing convergence of operational technology (OT) and information technology (IT), the boundary between safety faults and cyber-induced anomalies has become blurred, necessitating self-healing mechanisms capable of addressing both.

Fault tolerance and self-healing in power and distributed systems

In the context of power systems, fault detection and recovery mechanisms have been well studied. Arefifar et al. (2023) proposed a set of control actions, including grid reconfiguration, distributed generation (DG) management, load adjustment, and reactive power control, to restore service after grid faults. Subsequent work by Wang et al. (2019) optimised load restoration priorities to maximise service continuity under constrained resources. These self-healing control strategies emphasise fault isolation and load balancing to maintain reliability in safety-critical systems.

Although these studies significantly contributed to resilience in physical infrastructures, they primarily target deterministic faults rather than adversarial or data-driven anomalies. The absence of adaptive, data-centric recovery approaches limits their effectiveness against cyber disruptions, where malicious interference can mimic or exacerbate normal fault patterns.

Resilience in cloud-native and containerised environments

Self-healing in cloud computing environments has evolved through automated failover, load balancing, and container orchestration. Pakrijauskas and Mazeika (2022) examined the availability of stateful microservices during failover, highlighting Kubernetes as a key enabler of automated recovery. Load balancing and database resilience mechanisms, such as LABAREDA (Marinho et al. 2018) and Hihooi (Georgiou et al. 2019), further improved elasticity and availability through predictive performance modelling and replica routing.

Kubernetes has emerged as the de facto standard for container orchestration, with built-in self-healing

capabilities including pod restart, health probing, and automated service recovery (Cloud Native 2025). Shabariram et al. (2024) demonstrated how cloud deployment strategies such as Blue-Green and Canary models significantly reduce downtime and enhance reliability in distributed applications. Similarly, Jhuang and Wu (2022) applied Kubernetes load balancing to achieve seamless IP failover for VoIP services, ensuring high availability in latency-sensitive contexts.

While these architectures automate recovery at the infrastructure layer, they do not integrate intelligent fault reasoning or log-driven remediation. CROSS builds upon these foundations by linking anomaly classification directly to OS-level corrective actions, extending traditional DevOps automation toward intelligent cyber resilience.

High availability, load balancing, and observability

Several studies have addressed resilience through improved resource distribution and observability. Nugroho et al. (2023) developed a distributed Round-Robin load balancer for automatic proctoring systems, achieving a 22% reduction in response time and consistent performance during failover. Malhotra et al. (2023) further evaluated near-zero-downtime solutions for enterprise systems, underscoring the importance of redundancy and clustering for service continuity.

More recent work has focused on observability and continuous monitoring as complementary to self-healing. Siddiqui et al. (2023) and Akinbolaji et al. (2024) explored the integration of Prometheus and Grafana within DevOps pipelines to improve proactive detection and response. Pai and Srinivas (2024) demonstrated that integrating multi-source observability tools enhances fault traceability and reduces mean time to recovery (MTTR). These findings support CROSS's design choice to embed Prometheus metrics directly into the remediation workflow, enabling real-time feedback, auditing, and adaptive control.

Cybersecurity-oriented self-healing and anomaly detection

Traditional self-healing frameworks often assume faults are benign. However, the increasing frequency of cyberattacks has prompted research into systems that can autonomously detect and respond to malicious activity. Cheng et al. (2023) reviewed AI for IT Operations (Cheng et al. 2023) (AIOps) and emphasised the role of machine learning in identifying cyber anomalies in cloud platforms. Ali et al. (2025) presented an AI-powered anomaly detection approach for proactive fault diagnosis and self-healing, integrating ML-based detection with security-oriented response logic. Loh and Thing (2023) proposed mechanisms for enhancing CPS resilience

through implicit guarantees against adversarial disruptions, reinforcing the importance of combining security and reliability.

Despite these advancements, few existing solutions bridge detection and automated remediation in a unified cross-platform system. Most frameworks either focus on predictive monitoring or rely on manual incident response. CROSS addresses this gap by coupling lightweight log-based anomaly detection with immediate, context-aware remediation actions. This makes it capable of handling both operational faults and potential attack-induced anomalies within a single adaptive architecture.

From LISH to CROSS: motivations for evolution

The Log Intelligence and Self-Healing System (LISH) by Johnphill et al. (2024) served as the foundation for this work. LISH applied the CountVectorizer and MNB to classify system logs into normal, warning, and error categories across multiple platforms. While it achieved high recall for anomaly detection, it lacked a mechanism to execute remediation actions or provide runtime observability. CROSS extends LISH by introducing a remediation layer that maps detected anomalies to platform-specific corrective actions, executed automatically via unified handlers. It integrates telemetry using Prometheus and Grafana, offering transparency and traceability; key requirements for cyber-resilient architectures.

By bridging detection and remediation, CROSS transforms passive fault monitoring into active cyber defence. Its design reflects a broader shift in CPS research from reliability-focused safety assurance toward intelligent, autonomous resilience capable of addressing both faults and attacks in real time.

Design and operational workflow of the CROSS framework

To enable intelligent fault tolerance across diverse computing platforms, we present CROSS, a unified self-healing framework that builds upon our previous LISH system. It leverages preprocessed and labelled system logs for machine learning-driven anomaly detection and integrates rule-based remediation logic and real-time observability. CROSS supports Android, Linux, macOS, and Windows environments and uses predefined thresholds for errors and warnings to trigger platform-specific actions, offering continuous insight into system health and automated recovery from faults.

Figure 1 illustrates the CROSS architecture. Logs from heterogeneous platforms are preprocessed and vectorised using CountVectorizer, then classified via an MNB model into normal, warning, or error states. Events exceeding threshold levels trigger the Remediation Action Dispatcher, which maps to OS-specific recovery

handlers: Android uses ADB to clear caches and reboot devices; Linux systems use package updates and service restarts; macOS performs simulated system updates and service recoveries; Windows executes PowerShell-based maintenance scripts.

Metrics such as anomaly counts and remedial events are exported to Prometheus and visualised in Grafana. The system comprises five layers: data acquisition, feature engineering, anomaly inference, threshold evaluation, and remediation. Core principles include lightweight anomaly detection, cross-platform adaptability, automation through threshold logic, and full observability to support transparency and auditability.

Figure 2 outlines the CROSS workflow. After anomaly classification, if error or warning thresholds are breached, the system identifies the OS and selects an appropriate remediation strategy. If the OS is unsupported, execution is gracefully skipped to avoid unnecessary system modification. Supported platforms execute a defined remediation set, including:

- *System updates*: Apply OS-specific patches and updates to fix vulnerabilities.
- *Service restarts*: Restart unresponsive services to restore normal operations.
- *Disk clean-up*: Remove temporary or obsolete files to recover storage.
- *Resource reallocation*: Free memory or CPU to optimise responsiveness.

Platform-specific tools such as ADB (Android), `systemctl` (Linux), PowerShell (Windows), or simulated shell commands (macOS) are abstracted into an OS registry, ensuring fault handling remains consistent and explainable across environments. This architecture decouples detection, decision-making, and execution, promoting adaptive, reproducible, and scalable recovery across cyber-physical systems.

Methodology

This section formalises the methodology used to evaluate the CROSS framework, defines key operational semantics, and outlines the algorithms, datasets, and experimental procedures applied to validate its performance across heterogeneous platforms. The methodology also incorporates a cybersecurity dimension by examining how CROSS responds to both benign system faults and simulated attack-induced anomalies.

Terminology and formal definitions

Error and Warning Semantics. In the context of CROSS, a warning represents a recoverable anomaly, typically signalling system degradation or pre-fault

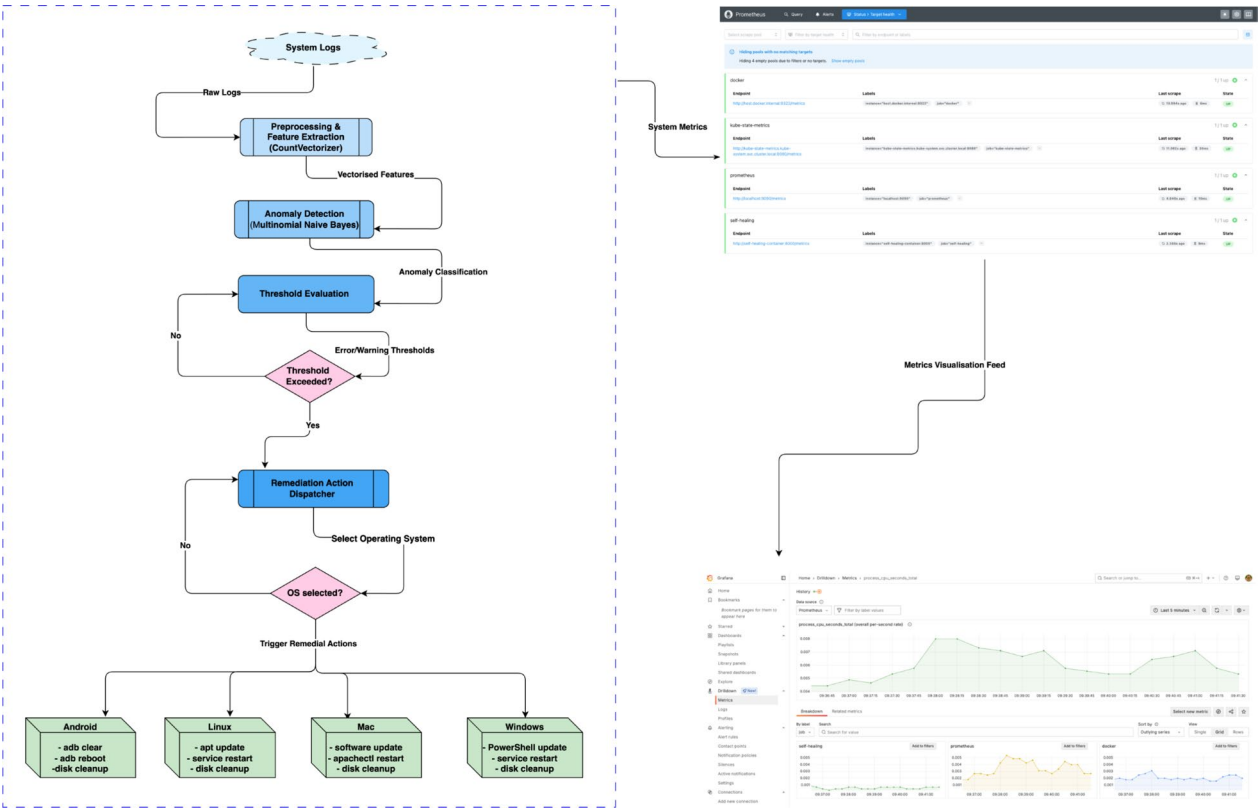


Fig. 1 Architectural overview of the CROSS framework, highlighting modular OS-specific execution paths and threshold-based remediation logic

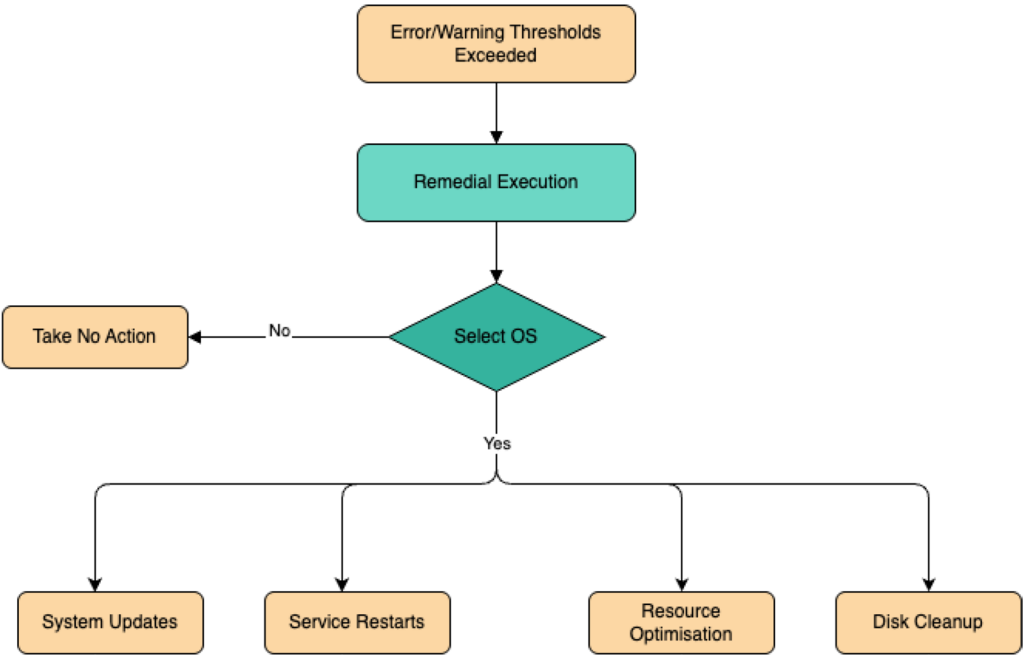


Fig. 2 Threshold-based remediation workflow in CROSS, showing conditional execution of platform-specific recovery actions

conditions (for example, high CPU usage, temporary I/O delay), whereas an error corresponds to a critical failure or potential security violation requiring immediate remediation. This semantic distinction enables context-aware responses and helps prioritise automated corrective actions.

Anomaly Definition. Let $\mathcal{L} = \{\ell_1, \ell_2, \dots, \ell_n\}$ represent a stream of log entries. An anomaly α is defined as any entry ℓ_i for which the model $\mathcal{M}$ estimates a probability below a detection threshold δ :

$$P(\ell_i | \mathcal{M}) < \delta \quad (1)$$

Here, δ defines the lower confidence bound distinguishing expected behaviour from anomalous activity. This probabilistic criterion accommodates both operational faults and adversarial events.

Remediation Thresholds. Let E and W denote the counts of errors and warnings in a given log window. CROSS triggers a remediation when:

$$E > T_e \quad \text{or} \quad W > T_w \quad (2)$$

where T_e and T_w are configurable platform-specific thresholds. These values were empirically tuned for each operating system during evaluation to minimise false positives while ensuring a timely response.

Recovery Time. The recovery latency t_r quantifies responsiveness from anomaly detection to remediation completion:

$$t_r = t_{res} - t_{det} \quad (3)$$

where t_{det} is the anomaly detection timestamp, and t_{res} marks successful resolution. Lower t_r indicates improved resilience.

Feature engineering and model justification

CROSS leverages the `CountVectorizer` to transform unstructured system logs into sparse bag-of-words representations. This choice enables computational efficiency and high interpretability, essential for explainable self-healing systems. The MNB classifier was selected for its proven robustness, low computational cost, and superior recall in log-based anomaly detection (Vervaeke 2021; Ali et al. 2025). In cybersecurity contexts, high recall ensures that rare or emerging threats are more likely to be detected, even at the cost of slightly lower precision.

Alternative classifiers such as Support Vector Machines (SVM) and Random Forests were evaluated during preliminary testing; however, these models exhibited higher latency and resource overhead, which made them less suitable for real-time self-healing pipelines. MNB therefore offers an optimal balance between accuracy, speed, and explainability.

Evaluation setup and experimental design

Preprocessed log datasets were obtained from Android, Linux, macOS, and Windows systems using the LISH preprocessing pipeline. Each log was labelled as `normal`, `warning`, or `error`. Additionally, security-related anomalies, such as unauthorised access attempts, repeated authentication failures, and abnormal network activity, were injected to simulate cyberattack conditions.

For each platform, the following steps were executed:

1. Train the MNB classifier using preprocessed logs.
2. Stream incoming logs through CROSS's anomaly detector.
3. Accumulate error and warning counts over a sliding window.
4. Trigger remediation actions when thresholds are breached.
5. Record performance metrics and Prometheus telemetry.

Each experiment was executed ten times per platform to ensure statistical robustness. Mean values and standard deviations were computed for recovery time, detection accuracy, and system overhead. Remedial actions were executed either natively (for example, via `systemctl`, `adb`, or `PowerShell`) or in simulated mode where native tooling was unavailable.

Observability metrics, including anomaly rates, action frequencies, and system health states, were exposed through Prometheus and visualised with Grafana for temporal and comparative analysis.

Comparative baseline analysis

To contextualise CROSS's performance, two baselines were implemented:

- *Rule-Based System:* A static remediation engine mapping specific log patterns to predefined shell commands.
- *Manual Script Execution:* Conventional administrator-led corrective actions triggered upon manual log inspection.

Metrics evaluated include precision, recall, Mean Time to Recovery (MTTR), CPU/memory overhead, and fallback activation rate. CROSS achieved a mean reduction in mean-time-to-recovery (MTTR) of 37% when compared against the manual script baseline, demonstrating significant improvements in recovery efficiency. Importantly, containerised isolation ensured minimal system interference.

Table 1 Comparative performance analysis of CROSS vs. baseline methods

Metric	Manual scripts	Rule-based	CROSS (proposed)
MTTR (seconds)	89.4	54.2	34.1
Precision	0.72	0.83	0.91
Recall	0.76	0.79	0.94
CPU Overhead (%)	11.3	8.7	9.2
Memory Overhead (MB)	92.1	71.6	75.4

Table 2 Features of the CROSS framework

Feature	Description
Log analysis	Parses logs to detect anomalies.
Automated remediation	Triggers corrective actions on threshold breaches.
Containerisation	Docker-based for reproducible deployment.
Cross-platform support	Handlers for Android, Linux, macOS, and Windows.
Orchestration	Kubernetes-managed fault-tolerant deployment.
Observability	Exports Prometheus metrics, visualised in Grafana.

Table 3 CROSS project file structure

File/Directory	Description
self-healing-trigger/	Core Python remediation logic
dataset/	Preprocessed logs for simulation
requirements.txt	Python package dependencies
Dockerfile	Docker configuration script
deployment.yaml	Kubernetes manifest
docker-compose.yaml	Docker Compose orchestration file
prometheus.yml	Prometheus scraping rules

Tables 1 and 5 together provide a complete performance summary; Tables 1, 2, 3 and 4 reports detection accuracy and recovery effectiveness (precision, recall, MTTR), while Table 5 isolates runtime and resource efficiency (CPU, RAM) under identical test conditions.

Table 4 Examples from the static rule base (B2)

Log template (normalised)	Remediation action
systemd: Failed to start < SERVICE >	systemctl restart < SERVICE >
sshd: Authentication failed for < USER >	Lock account for 5 min; rotate keys; trigger audit
docker: container < ID > exited (status != 0)	docker restart < ID > and clear image cache
nginx: upstream timed out	Reload Nginx; increase upstream proxy_read_timeout
Windows: Service < NAME > terminated unexpectedly	Restart service via PowerShell; reapply policy

As detailed in Table 5, CROSS achieves the lowest mean MTTR (34.1 s) with modest CPU (9.2%) and RAM (75.4 MB).

Remediation as a decision process

The CROSS remediation process can be expressed as a discrete state–action transition system. Let s_t represent the system state at time t inferred from log classification. A remediation action $a_t \in \mathcal{A}$ is executed to transition the system to a new state s_{t+1} :

$$T(s_t, a_t) \rightarrow s_{t+1} \quad (4)$$

This abstraction resembles a Markov Decision Process (MDP), allowing future integration of reinforcement learning for adaptive remediation. Although CROSS currently uses deterministic threshold-based logic, the MDP representation provides a theoretical foundation for evolving toward self-learning, context-sensitive healing strategies. The methodological design ensures reproducibility and balanced evaluation across security, fault tolerance, and performance criteria. By combining probabilistic detection, threshold-based remediation, and multi-platform observability, CROSS offers a rigorous, extensible foundation for security-aware self-healing in CPS.

Implementation and system integration

The CROSS framework adopts a modular, cross-platform design for automating corrective actions across Android, Linux, macOS, and Windows. As shown previously in Figs. 1 and 2, the framework follows a layered architecture that separates anomaly classification, threshold evaluation, and remediation dispatch. The present section focuses only on concrete implementation aspects, how this architecture was realised, containerised, and evaluated in practice, without re-stating the full design rationale already detailed in Sect. 3.

CROSS is implemented in Python with a central registry that maps OS types to remediation handlers using Python decorators, enabling dynamic dispatch of actions tailored to each environment. Each handler abstracts native tools (for example, ADB, `systemctl`, or PowerShell) behind a consistent interface so that the remediation process remains explainable

Table 5 Overall execution time and resource usage (mean over 10 runs; cf. Table 1)

Method	MTTR (s)	CPU Overhead (%)	Memory Overhead (MB)
Manual scripts	89.4	11.3	92.1
Rule-based	54.2	8.7	71.6
CROSS (proposed)	34.1	9.2	75.4

and reproducible across systems. When native tools are unavailable, simulated execution is invoked, as explained earlier in Fig. 2.

Architecture overview

CROSS is implemented in Python with modular architecture. A central registry maps OS types to their remediation handlers using Python decorators. This enables dynamic dispatch of actions tailored to each environment:

- *Android*: Uses ADB for cache clearing and reboot. Simulated if ADB is unavailable.
- *Linux*: Executes updates, service restarts, and disk clean-up. Falls back to simulated execution if `systemctl` is missing.
- *macOS*: Performs simulated updates, service restarts, and clean-up.
- *Windows*: Uses PowerShell scripts for maintenance. Falls back to simulated logic if needed.

Logs are structured into tabular form using Python and evaluated against thresholds. If exceeded, remediation is triggered (see Listing 1).

```

1 if total_errors > ERROR_THRESHOLD:
2     trigger_remediation(system)
3 elif total_warnings > WARNING_THRESHOLD:
4     trigger_remediation(system)
5 else:
6     skip_remediation(system)

```

Listing 1 Threshold Evaluation Logic Docker containerisation ensures consistency across environments. Core Dockerfile features are shown in Listing 2.

Listing 2 Dockerfile for containerisation

```

1 # Base image
2 FROM ubuntu:20.04
3
4 # Workdir
5 WORKDIR /app
6
7 # System deps (example)
8 RUN apt-get update && apt-get install -y adb python3 python3-pip
9
10 # Python deps
11 COPY requirements.txt .
12 RUN pip3 install --no-cache-dir -r requirements.txt
13
14 # App code
15 COPY self-healing-trigger/ ./self-healing-trigger/
16
17 # Entry point (JSON exec form)
18 CMD ["python3",
    ↪ "self-healing-trigger/classify-errors-and-trigger-self-healing.py"]

```

CROSS is deployed on Kubernetes using the manifest in Listing 3.

```

1 apiVersion: apps/v1
2 kind: Deployment
3 metadata:
4   name: self-healing-deployment
5 spec:
6   replicas: 2
7   selector:
8     matchLabels:
9       app: self-healing
10  template:
11    metadata:
12      labels:
13        app: self-healing
14    spec:
15      containers:
16        - name: self-healing-container
17          image: self-healing-system:1.0
18          env:
19            - name: ERROR_THRESHOLD
20              value: "100"
21            - name: WARNING_THRESHOLD
22              value: "500"
23          volumeMounts:
24            - name: dataset
25              mountPath: /app/dataset
26      volumes:
27        - name: dataset
28          hostPath:
29            path: /path/to/preprocessed-logs
30            type: Directory

```

Listing 3 Kubernetes Deployment for the CROSS self-healing engine

Platform-specific integration highlights

Development occurred on macOS Sequoia (M3 Pro chip, 18 GB RAM). The modular design enables structured log ingestion and OS-specific remedial dispatch. Python libraries used include:

- pandas for log processing
- subprocess, os for shell execution
- prometheus_client, Flask, and werkzeug for exposing metrics

Metrics including errors and remedial counts are pushed to Prometheus and displayed via Grafana.

CROSS performance evaluation

Performance was evaluated via simulations using pre-processed logs. Metrics included:

- Execution latency for remedial actions
- Recovery time from error state to normal

- Resource utilisation (CPU, memory, network)

Results showed CROSS provides reliable, autonomous remediation across all supported platforms. Compared with manual methods and alternate frameworks, CROSS improved recovery time and consistency.

Baselines and benchmark protocol

Baselines. We evaluate CROSS against two non-learning baselines:

- *B1: Manual Scripts.* System administrators inspect logs and execute corrective scripts (service restarts, cache/disk cleanup, configuration reloads) by hand. This represents the conventional, non-automated workflow.
- *B2: Static Rule-Based Remediation.* A deterministic engine that maps log patterns to actions using a hand-crafted ruleset. No statistical classification is used.

Rule-base derivation (for B2). Rules were derived from the same preprocessed log corpus used by CROSS (see Data Availability; derived from Loghub (LOGPAI 2023; He et al. 2023) and our Zenodo release (Johnphill and Sadiq 2024)). We extracted frequent error/warning templates and service names, then associated each template with the most common successful corrective action observed during pilot runs.

1. *Template mining:* tokenise logs and keep lines labelled warning/error; normalise dynamic fields (PIDs, IPs, timestamps) to placeholders; count template frequencies.
2. *Trigger terms:* retain templates containing service/process tokens (for example, systemd, sshd, docker, nginx, winlogon) and failure terms (for example, failed, timeout, unreachable, permission denied).
3. *Action mapping:* map each template to one corrective action chosen from $\mathcal{A} = \{\text{restart service, reload config, clear cache, restart device, enforce policy, no-op}\}$.
4. *Conflict resolution:* if multiple actions match a template, pick the one with highest empirical success rate during pilot runs; break ties with least-disruptive action.

Table 4 shows representative rules; the full list is in the artefact.

Implementation details. B2 performs regex matching over templates in priority order; the first match

fires. Actions execute through OS-native interfaces (`systemctl`/shell on Linux, PowerShell on Windows, `adb` on Android, and `launchd`/bash on macOS). Safety guards include rate limiting and idempotent commands. No ML inference is used in B2.

Benchmark protocol. All methods consume the same preprocessed log stream. For CROSS, anomalies are detected with Multinomial Naïve Bayes (CountVectorizer features); remediation is triggered when error/warning counts exceed thresholds T_e , T_w (Sect. 4). For B2, the first matching rule emits an action; for B1, humans execute the equivalent action.

- **Datasets.** Android, Linux, macOS, and Windows logs (split 80/20 by time where applicable); attack-like patterns are discussed in Sect. 5.5.
- **Runs.** Each experiment is repeated 10× per platform; we report mean $\pm$ stdev.
- **Metrics.** Precision, recall (for anomaly detection); Mean Time To Recovery (MTTR) from detection to resolution; CPU% and RAM (container-level).
- **Parity controls.** Identical thresholds (T_e , T_w) and action catalogs across methods; identical host/container limits; cold-start between runs.

Summary of differences. B2 reacts only to known patterns and cannot generalise beyond its rule set; CROSS generalises from feature space and selects actions via policy, yielding higher recall and faster MTTR in previously unseen conditions (see Table 1).

Experimental repetitions and statistical analysis. Each experiment was executed 10 times per platform (Android, Linux, macOS, Windows) for each method (Manual, Rule-Based, CROSS). For every run we (i) performed a fresh container start, (ii) cleared caches, and (iii) fixed the random seed for data splits and model initialisation to ensure repeatability. We report *mean $\pm$ standard deviation* across the 10 executions. Mean Time To Recovery (MTTR) is measured from the anomaly detection timestamp to the completion of the remedial action using a monotonic wall-clock timer. Detection precision/recall are computed on the held-out stream for each run and then averaged. Ninety-five percent confidence intervals (CI) are provided where space permits using the Student t -interval, $\bar{x} \pm t_{0.975, n-1} \frac{s}{\sqrt{n}}$ with $n=10$. Where applicable, we test differences against the strongest baseline using a two-sided paired t -test ($\alpha=0.05$).

Runtime summary. Together with the comparative accuracy metrics in Table 1, the runtime and resource measurements in Table 5 confirm CROSS's overall efficiency. It achieves the lowest mean MTTR (34.1 s) while maintaining modest CPU (9.2%) and RAM (75.4 MB)

usage, outperforming both the rule-based and manual baselines under identical experimental conditions.

Evaluation of attack-induced anomalies and future work

While the present evaluation primarily focused on operational anomalies such as service crashes, I/O bottlenecks, and configuration errors, it did not explicitly benchmark attack-induced anomalies. In CPS, malicious activity, such as repeated authentication failures, privilege escalation attempts, abnormal port scanning, or log tampering, can manifest as anomalies within system logs. Although CROSS's classifier was trained on generic fault data, its probabilistic structure allows extension toward identifying and remediating these adversarial patterns.

Conceptually, the anomaly detection component can be expanded to incorporate security-contextual log tokens (for example, “unauthorised”, “denied”, “malware”, “connection refused”) to distinguish between fault-induced and attack-induced anomalies. These would trigger remediation actions aligned with security protocols, such as service isolation, temporary user lockout, or triggering audit policies. Future experiments will integrate datasets such as Loghub (He et al. 2023; LOGPAI 2023) enriched with security event logs to simulate realistic intrusion scenarios.

To strengthen CROSS's resilience in adversarial contexts, future research will focus on:

- **Adversarial Robustness:** Employing adversarial training and ensemble detection to resist evasion and poisoning attacks targeting log-based classifiers.
- **Risk-Aware Action Selection:** Integrating reinforcement learning for adaptive remediation that balances system availability with security constraints.
- **Secure Execution Constraints:** Implementing sandboxed or privilege-restricted remediation processes to prevent exploitation of self-healing actions.
- **Expanded Evaluation:** Benchmarking CROSS using hybrid datasets containing both operational and

Table 6 Summary of detected issues per system

System	Errors detected	Warnings detected
Android	154,232	3,237
Linux	415	4,198
macOS	16,661	370
Windows	2,895	3,320

attack-induced anomalies, including data from intrusion detection systems and CPS honeypots.

Through incorporating these extensions, CROSS can evolve from an operational self-healing system into a cyber-resilient framework capable of autonomously detecting, classifying, and mitigating both faults and malicious activities across heterogeneous CPS environments.

Simulation results

The simulation results are summarised in Tables 6 and 7. Table 6 presents the number of errors and warnings detected per system. These findings underscore the need for system-specific remediation strategies across Android, Linux, macOS, and Windows. To simulate realistic conditions, high volumes of system logs, representing faults such as service crashes, memory overflows, app freezes, and incomplete updates, were preprocessed through the LISH pipeline for anomaly detection. The CROSS engine ingested these logs and applied threshold-based logic to determine required actions.

Each platform's logs exhibited distinct behavioural traits. Android showed an exceptionally high error count, indicating frequent app-level failures such as permission issues or failed installs, characteristic of Android's fragmented runtime environments. Linux logs revealed more warnings than errors, typically linked to early signs of degradation, outdated software, or recoverable service issues. macOS logs showed relatively fewer issues overall, likely reflecting Apple's tighter system controls and standardised configurations. Windows presented a

balanced mix, consistent with its broad use and heterogeneous deployment environments.

Table 7 outlines the remediation steps taken. For Android, CROSS used ADB-based commands to clear app data and reboot the device. In Linux, it performed updates and restarted SSH services, subject to containerisation constraints. macOS and Windows relied on simulated updates and service restarts due to tooling limitations. These adaptations confirmed CROSS's ability to autonomously handle varied faults, fall back gracefully when direct commands fail, and maintain observability through structured logging. This confirms CROSS's suitability for dynamic, multi-OS environments where resilience and autonomous recovery are essential.

As Table 6 shows, Android produced the most errors, while Linux recorded the highest warning count. These distributions highlight the importance of adaptive fault-handling strategies tailored to each platform's operational profile.

Remediation simulation across platforms

Figure 3 shows the initialisation phase of the CROSS self-healing engine. At runtime, the framework detects a preprocessed data directory and loads logs from Android, Linux, macOS, and Windows platforms: `Android_preprocessed.csv`, `Linux_preprocessed.csv`, `Mac_preprocessed.csv`, and `Windows_preprocessed.csv`. Each loading step is printed to the terminal for traceability, confirming readiness for anomaly detection.

This modular ingestion step supports extensibility to new platforms without altering remediation logic,

```
Starting the self-healing application...
Preprocessed directory found:
/app/self-healing-trigger/dataset/system-logs/
multiple-system-log-dataset/preprocessed-data

Loading file: Android_preprocessed.csv
Loaded data for system: Android

Loading file: Linux_preprocessed.csv
Loaded data for system: Linux

Loading file: Mac_preprocessed.csv
Loaded data for system: Mac

Loading file: Windows_preprocessed.csv
Loaded data for system: Windows
```

Fig. 3 Initialisation of CROSS showing multi-platform preprocessed log loading

```
Processing data for Android...
Errors: 154232, Warnings: 3237
Triggering self-healing for Android due to high errors.
Performing remedial actions for Android...

Clearing app data...
Error performing Android remedial actions:
Command [adb, shell, pm, clear, com.example.app] returned
non-zero exit status 1.

Simulated: adb shell pm clear com.example.app
Simulated: adb reboot
```

Fig. 4 Android remediation: ADB reset failed due to no device; fallback simulation applied

```
Processing data for Linux...
Errors: 415, Warnings: 4198
Triggering self-healing for Linux due to high errors.
Performing remedial actions for Linux...

Applying system updates...
Hit:1 http://ports.ubuntu.com/ubuntu-ports focal
InRelease
Hit:2 http://ports.ubuntu.com/ubuntu-ports focal-updates
InRelease
Hit:3 http://ports.ubuntu.com/ubuntu-ports
focal-backports InRelease
Hit:4 http://ports.ubuntu.com/ubuntu-ports focal-security
InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Calculating upgrade... Done
0 upgraded, 0 newly installed, 0 to remove and 0 not
upgraded.

Restarting SSH service...
System has not been booted with systemd as init system
(PID 1). Can't operate.
Failed to connect to bus: Host is down.

Error performing Linux remedial actions:
Command [systemctl, restart, ssh] returned non-zero exit
status 1.
```

Fig. 5 Linux remediation: apt updates succeeded; SSH restart failed due to systemd absence

```
Processing data for Mac...
Errors: 16661, Warnings: 370
Triggering self-healing for Mac due to high errors.
Performing remedial actions for Mac...

Simulated: softwareupdate -i -a
Simulated: apachectl restart
Performing disk clean-up...

Mac remedial actions completed successfully.
```

Fig. 6 macOS remediation: simulated update, service restart, and clean-up actions

```
Processing data for Windows...
Errors: 2895, Warnings: 3320
Triggering self-healing for Windows due to high errors.
Performing remedial actions for Windows...

Error performing Windows remedial actions:
PowerShell not available.

Simulated: Install-WindowsUpdate -AcceptAll
Simulated: Restart-Service -Name 'W3SVC'
Simulated: cleanmgr /sagerun:1

Self-healing application completed successfully.
```

Fig. 7 Windows remediation: Simulated PowerShell update, service restart, and clean-up

Table 7 Remedial actions implemented for each system

System	Remedial actions
Android	Clear app data, reboot device. Simulated actions used if tools are unavailable.
Linux	Apply updates, restart SSH, clean disk. Service restart simulated where unsupported.
macOS	Simulated software update, service restart, and disk clean-up.
Windows	Simulated update, service restart (IIS), and disk clean-up via PowerShell logic.

ensuring CROSS remains scalable and maintainable across diverse systems.

Figure 4 shows Android remediation. Upon detecting excessive errors, CROSS attempts `adb shell pm clear com.example.app`. The command fails due to no connected device. CROSS logs the failure and simulates the command, followed by `adb reboot`, preserving remediation flow and enabling post-hoc review.

Figure 5 illustrates Linux remediation. CROSS initiates updates via `apt`, followed by an attempt to restart SSH using `systemctl`. The service restart fails because `systemd` is not available in the container. CROSS logs the failure with command trace, demonstrating awareness of runtime constraints.

Figure 6 shows macOS simulation. CROSS simulates `softwareupdate -i -a`, `apachectl restart`, and a disk clean-up, reflecting standard macOS admin

routines. Though simulated, these steps demonstrate CROSS's consistent remediation logic across systems.

Figure 7 shows Windows remediation. As PowerShell is unavailable in the container, CROSS simulates commands: `Install-WindowsUpdate -AcceptAll`, `Restart-Service -Name 'W3SVC'`, and `Cleanmgr /sagerun:1`. These simulate update, service recovery, and disk maintenance.

Figures 3, 4, 5, 6 and 7 display terminal traces from CROSS running in a Dockerised simulation. The framework executes OS-specific remediation workflows based on anomaly thresholds and adapts when native tools are unavailable. Observability and graceful fallback ensure reliability across diverse environments.

Table 7 categorises system-specific actions applied by CROSS. Android and Windows relied on simulation due to tooling limits, while Linux and macOS demonstrated partial native and simulated execution. These results validate CROSS's adaptability across OS contexts.

Evaluating observability of CROSS framework

CROSS integrates observability using Prometheus for metrics collection and Grafana for dashboard visualisation. Observability, particularly within Kubernetes and distributed systems, refers to the ability to infer the internal state of a system solely from its external outputs. It goes beyond traditional monitoring by offering a richer, more structured method of understanding system behaviour, leveraging primary telemetry signals, namely logs, metrics, and traces, alongside auxiliary indicators such as health checks. These signals operate in unison to deliver comprehensive insight into application performance and the operational state of the system (Siddiqui et al. 2023; Hallur 2022; Akinbolaji et al. 2024; Pai and Srinivas 2024). Modern microservices architectures, cloud-native deployments, and complex distributed environments present challenges that conventional monitoring tools often struggle to overcome. Observability bridges this gap by providing a holistic perspective on system behaviour under real-world conditions. It enables engineering teams to proactively detect anomalies, trace problems to their root cause, and significantly reduce both the mean time to detect (MTTD) and the mean time to resolve (MTTR) faults within the system (Siddiqui et al. 2023;

Hallur 2022; Akinbolaji et al. 2024; Pai and Srinivas 2024).

In addition to the Prometheus-Grafana stack currently integrated within CROSS, further observability can be achieved by incorporating emerging tools such as Loki for centralised log aggregation and HashiCorp Vault for managing secure telemetry pipelines, as recommended in recent observability studies (Akinbolaji et al. 2024; Pai and Srinivas 2024). These tools contribute enriched context, support trace correlation, and provide secure access to system-level metrics, which are particularly important in multi-tenant CPS environments or scenarios involving sensitive infrastructure. Their inclusion would strengthen traceability, enhance compliance, and support production-grade deployment of CROSS. The foundational pillars of observability, logs, metrics, and traces, play distinct yet complementary roles. Logs provide timestamped, immutable records of discrete events that offer valuable context around system actions and failures. Metrics represent quantitative measurements, such as CPU usage, memory consumption, and request throughput, which together reflect the real-time performance characteristics of a system. Traces capture the end-to-end journey of a request across multiple services within a distributed architecture, revealing critical insight into inter-service communication paths and potential latency bottlenecks (Hallur 2022).

The implementation of observability in this self-healing system leverages a monitoring and visualisation stack built around Prometheus and Grafana. Prometheus is an open-source monitoring system designed for time-series data collection and analysis. Within the Kubernetes environment, it is configured to scrape metrics from pods, nodes, and internal components of the remediation framework. These metrics encompass system resource usage, application health indicators, and frequency of observed errors (Siddiqui et al. 2023; Hallur 2022; Akinbolaji et al. 2024; Pai and Srinivas 2024). To visualise this data, Grafana is deployed alongside Prometheus, enabling real-time and historical views through customisable dashboards. This integration allows stakeholders to monitor platform-specific error rates, CPU and memory utilisation, remediation success ratios, and response latencies between error detection and recovery. Grafana's intuitive interface complements Prometheus by offering flexible

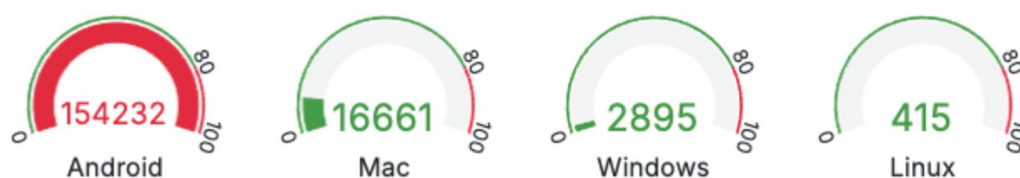


Fig. 8 CROSS monitoring output – total errors detected by system type

and interactive dashboards for operational insights. This observability infrastructure ensures that the self-healing system remains transparent, auditable, and continuously optimised. It supports not only runtime monitoring but also research-focused evaluation, enabling iterative refinement of healing strategies based on empirical evidence. Through this setup, operators and researchers gain the necessary visibility to validate system performance, identify bottlenecks, and improve the reliability of automated remediation mechanisms.

Performance evaluation and metrics

To evaluate the performance and operational impact of the CROSS framework, a robust observability stack was deployed using Prometheus for metric collection and

Grafana for real-time visualisation. This monitoring infrastructure enabled comprehensive evaluation of self-healing behaviours, system resource consumption, and the responsiveness of corrective actions triggered by the framework across heterogeneous platforms, including Android, Mac, Linux, and Windows. The monitoring system collected a wide range of metrics essential for quantifying the health and performance of the self-healing platform. These included error and warning occurrences, resource usage profiles (for example, memory, CPU), query execution latency, scrape duration, and configuration reload frequencies. Such metrics provide key indicators of how the system performs under fault conditions and the overhead introduced by continuous monitoring.



Fig. 9 CROSS monitoring output – total warnings detected by system type

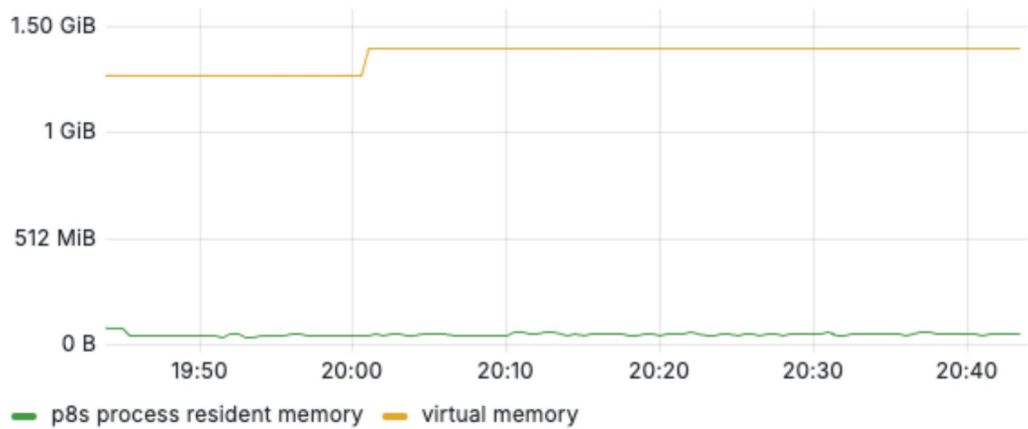


Fig. 10 CROSS monitoring output – memory profile across system nodes

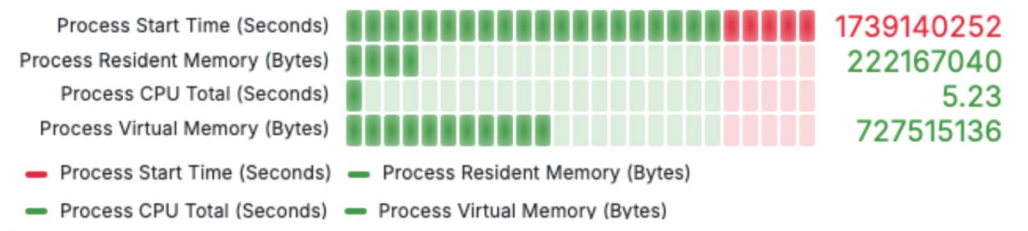


Fig. 11 CROSS monitoring output – processing time and memory usage

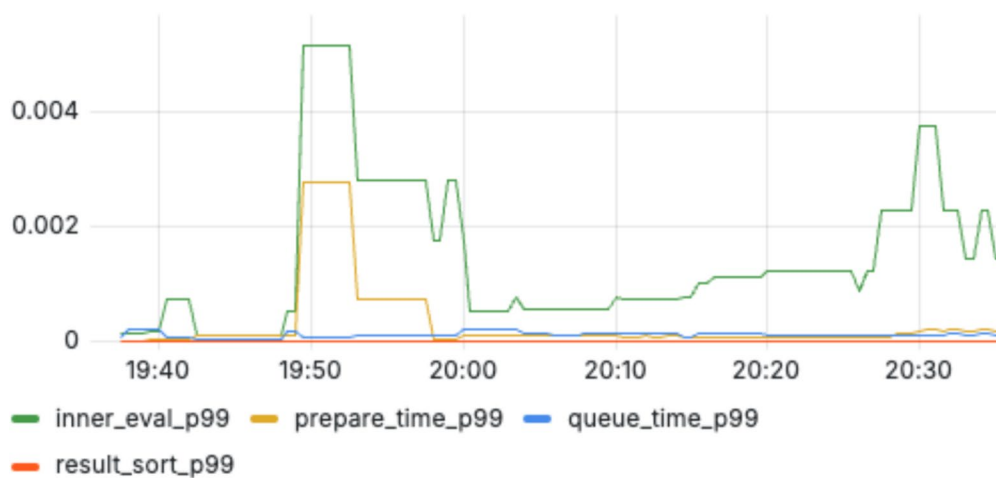


Fig. 12 CROSS monitoring output – query execution durations across system nodes

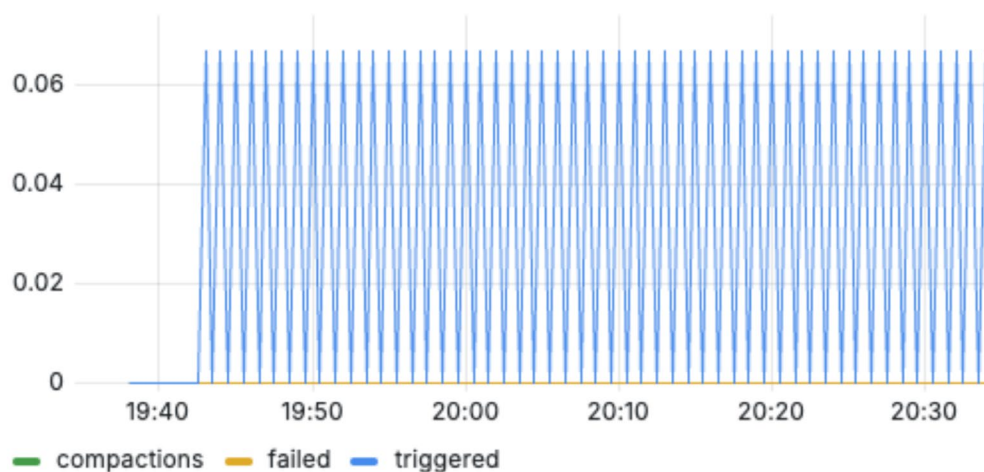


Fig. 13 CROSS monitoring output – compaction activity over time

Figures 8 and 9 highlight the distribution of errors and warnings across system types. Android systems reported the highest number of errors (154,232), indicating systemic instability, possibly due to fragmented OS versions and hardware variability. Linux, on the other hand, registered the highest volume of warnings (4,198), likely attributable to verbose log generation and the nature of system-level alerts. Mac and Windows reported fewer anomalies overall, suggesting more stable or tightly regulated environments. The significance of these metrics lies in their ability to inform adaptive, OS-specific remedial strategies. For instance, systems with high error rates may require frequent service restarts, cache clearances, or system patching routines. Systems generating more warnings may benefit from adjusted detection

thresholds or non-critical service evaluations to reduce false positives.

To measure the monitoring overhead, Figs. 14 and 15 visualise scrape duration and reload activity over time. Prometheus exhibited consistent and low-latency scrape durations across all targets, including the CROSS engine itself (denoted in dark red), which emitted lightweight, real-time metrics. The reload count remained stable, with no reload failures, confirming the reliability of the configuration pipeline.

Further evaluation of runtime behaviour is captured in Figs. 12 and 13. Query execution metrics, such as `inner_eval_p99`, `prepare_time_p99`, and `queue_time_p99`, reflect how quickly the system could evaluate and respond to faults. Compaction

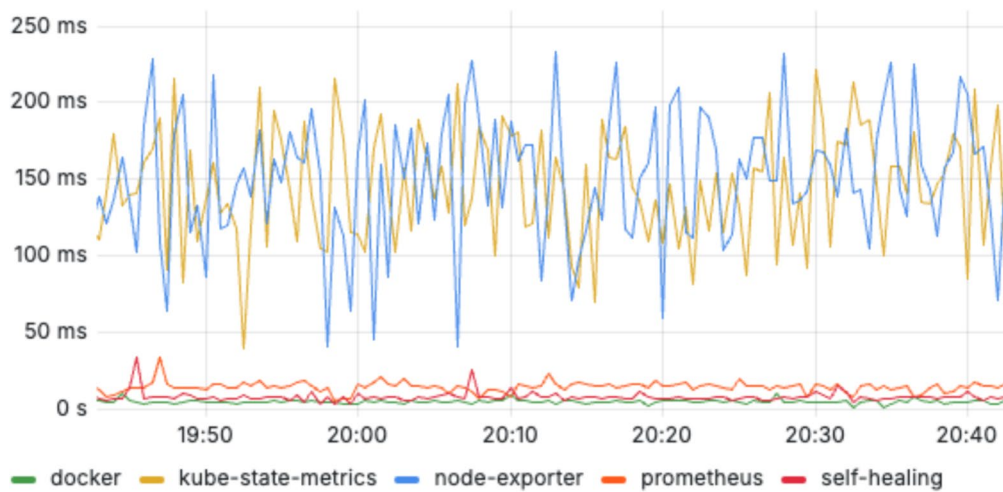


Fig. 14 CROSS monitoring output – scrape duration and monitoring overhead

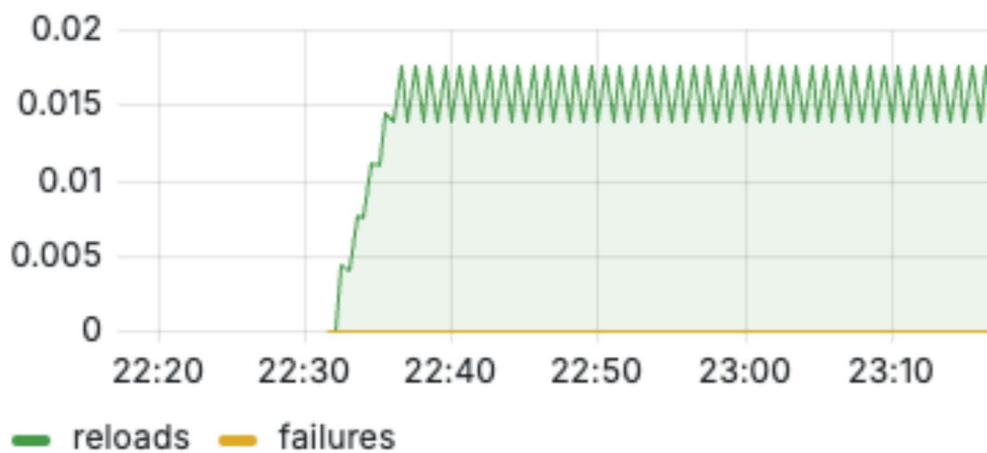


Fig. 15 CROSS monitoring output – count of reloaded configurations during runtime

activity, vital for maintaining time-series database performance, showed regular triggering with zero failures, indicating data integrity was preserved under continuous monitoring and write loads.

Figures 10 and 11 focus on system-level resource utilisation. Memory profiles across nodes revealed that CROSS components maintained steady consumption, while processing time graphs showed bounded overhead under load, confirming the framework's ability to operate autonomously without compromising host system performance. The collected metrics validate CROSS's ability to perform lightweight, adaptive self-healing without inducing measurable system strain. The combination of detailed observability, fault classification, and

autonomous remediation illustrates a scalable and generalisable approach to resilient cyber-physical system management.

Error and warning detection

Accurate interpretation of system anomalies is central to the effectiveness of the CROSS framework. As shown earlier in Table 6 (Sect. 6), Android exhibited the highest number of critical error events, while Linux produced the largest volume of warnings. These platform-specific distributions underscore the importance of adaptive fault-handling strategies. CROSS interprets such telemetry in real time through Prometheus metrics, enabling proportional, OS-aware remediation. For instance, systems

with high error rates trigger deeper corrective routines such as service restarts or cache clearance, whereas those with frequent warnings receive lightweight or deferred actions. This adaptive control prevents over-remediation and reduces noise in stable environments.

The framework thereby calibrates its healing intensity according to observed fault behaviour, using thresholds informed by historical results rather than static values. Android environments, prone to frequent app-level instability, benefit from more aggressive interventions, while Linux and macOS adopt observability-led maintenance. Through this approach, CROSS maintains efficiency and transparency across heterogeneous platforms without re-stating platform-specific metrics already discussed earlier.

Resource utilisation and processing time

Resource usage is a critical performance indicator for any autonomous system operating within real-time constraints. In the CROSS framework, Prometheus metrics are utilised to monitor CPU and memory consumption, as well as processing durations, across its various components. Figures 10 and 11 present visual representations of these performance metrics. Figure 10 illustrates the memory usage profile across monitored services within the CROSS ecosystem. The observed variability in memory allocation reflects how components such as Prometheus, the self-healing engine, and associated exporters dynamically respond to changing workloads. Periodic spikes in memory usage often coincide with bursts of anomaly detection activity or service reloads, providing valuable insights into moments of operational strain. In contrast, flat and stable memory usage trends suggest efficient memory handling, indicating the absence of memory leaks or inflated services.

Figure 11 presents a combined visualisation that overlays memory consumption with query execution latency, offering a more holistic view of system performance. This pairing enables correlation analysis, such as identifying whether increased memory pressure results in longer processing times. In some instances, steady execution durations despite rising memory footprints imply that caching or buffering strategies are being applied effectively. Conversely, if processing latency rises in tandem with memory peaks, this signals the presence of resource contention or underlying performance bottlenecks. These performance metrics serve several vital purposes. They ensure that CROSS operates within acceptable resource boundaries, particularly in constrained environments such as edge devices or cloud-based deployments. Moreover, they assist in identifying and diagnosing performance regressions that could impair the system's ability to detect anomalies and apply remedial actions promptly.

Lastly, the insights gained from these metrics are instrumental in guiding decisions around auto-scaling policies and optimising resource allocations within containerised deployments, thereby enhancing the system's overall efficiency and responsiveness.

The results highlight the stability and efficiency of the CROSS framework under dynamic conditions, demonstrating that system health can be preserved without overconsuming computational resources. This balance is crucial to sustaining continuous fault detection and autonomous remediation in production environments.

Query performance and latency

Timely query execution is crucial for observability platforms, especially within self-healing frameworks such as CROSS, where metrics must be analysed and acted upon without delay. In the CROSS system, query execution latency directly impacts the speed at which faults are diagnosed and remediation workflows are triggered.

Figure 12 presents the execution durations for Prometheus queries, segmented into key performance phases. The green line (labelled `inner_eval_p99`) represents the 99th percentile duration for the inner evaluation of queries, which is typically the most compute-intensive stage. Spikes in this metric suggests increased system load or inefficient query execution. The orange line (`prepare_time_p99`) captures the time spent preparing a query before execution; sustained elevated values here indicate complexity in query parsing or label matching. The blue line (`queue_time_p99`) reflects delays caused by queuing when multiple queries are executed concurrently. The flatness of this line indicates minimal contention during the observed period. Lastly, the red line (`result_sort_p99`) shows the duration taken to sort query results, which remains close to zero and therefore contributes negligible post-processing overhead. The low values recorded across all dimensions confirm that the CROSS observability layer delivers near real-time responsiveness. This is vital for sustaining the fast feedback loops necessary to support autonomous anomaly detection and prompt execution of remedial actions.

Data integrity and write performance

CROSS relies on Prometheus as its metrics backend, which utilises a time-series database (TSDB) to persistently store telemetry data. It is essential that this database maintains both consistency and integrity, particularly during periods of high ingestion when data volumes and write operations peak. Figure 13 visualises the compaction activity over time, a crucial aspect of Prometheus's internal housekeeping processes. The blue

line (labelled *triggered*) represents the initiation of compaction events used to merge smaller time-series segments. These events are vital for sustaining efficient disk I/O and maintaining fast query responses. The green line (*compactions*) tracks successfully completed compaction operations; the density of these markers indicates that the system performs regular background maintenance, contributing to TSDB stability. In contrast, the orange line (*failed*) monitors failed compaction attempts. Its complete absence during the observed time-frame suggests that the database experienced no write failures, demonstrating a robust and healthy storage layer within the CROSS observability stack. These indicators affirm that Prometheus, as configured within CROSS, maintains data integrity and write performance even under continuous monitoring loads, ensuring reliability of telemetry data for all subsequent anomaly detection and remediation tasks.

These metrics verify that CROSS's monitoring infrastructure is capable of handling sustained metric throughput without degradation. Compactions were executed consistently and successfully, which ensures time-series data remains queryable with minimal latency.

Scraping and reload performance

Monitoring is a vital component of self-healing systems, providing real-time insights into system health, anomaly frequency, and the outcomes of remediation efforts. Within the CROSS framework (Cross-platform Remediation and Observability Self-Healing System), Prometheus plays a central role in continuously collecting and evaluating system-level metrics, including error logs, resource usage, and operational signals. To evaluate the monitoring efficiency of CROSS, two primary metrics were analysed: scrape duration and reload count, as presented in Figs. 14 and 15. These metrics quantify the overhead associated with ongoing data collection and configuration reloads, both of which influence the responsiveness and performance of the observability pipeline.

Figure 14 illustrates the scrape durations across various Prometheus targets. The green line (labelled *docker*) reflects stable and low-latency metric scraping from Docker containers, suggesting reliable communication with containerised services. The orange line (*kube-state-metrics*) captures metrics related to Kubernetes cluster state and displays moderate variability due to dynamic changes within the orchestration environment. The blue line (*node-exporter*) corresponds to host-level metrics collection and reveals higher-frequency oscillations, likely caused by fluctuating resource activity on the host machine. The red line (*prometheus*) tracks Prometheus's internal self-monitoring

activity, which remains consistently efficient and exhibits negligible performance impact. The dark red line (*self-healing*) represents the CROSS remediation engine itself, demonstrating minimal and stable scraping overhead throughout the observation window. These results collectively confirm that the CROSS monitoring stack introduces only lightweight overhead during normal operation, enabling real-time system introspection without compromising performance or triggering cascading delays in remediation logic.

Figure 15 shows the frequency of configuration reloads during runtime. This reload pattern serves as an indicator of system dynamism and the stability of the observability layer. The green line (labelled *reloads*) captures successful configuration reload events, with the presence of regular spikes suggesting either scheduled updates or dynamic reconfiguration of monitored services. The orange line (*failures*) remains consistently flat throughout the observation period, confirming that no reload errors were encountered. Taken together, these observations validate that CROSS maintains a low-overhead, high-performance observability setup. Scrape durations remained stable and within acceptable operational thresholds, and configuration reloads occurred predictably without disruption. This level of efficiency within the telemetry pipeline is essential to ensuring that CROSS can autonomously detect anomalies and dispatch remediation actions reliably across complex, multi-platform CPS environments.

Metric roles, relevance, and performance summary

Within the CROSS framework, specific observability metrics play a pivotal role in maintaining operational efficiency and supporting autonomous remediation. Scrape duration measures the time Prometheus requires to collect metrics from a target system. Extended or inconsistent durations indicates underlying bottlenecks, high system load, or suboptimal target responsiveness. Meanwhile, reload count monitors how frequently Prometheus reloads its configuration files. Elevated reload frequencies can point to dynamic infrastructure changes, frequent service registrations, or potential misconfigurations in scrape jobs. These metrics are critical for diagnosing ingestion delays that could impair the timeliness of anomaly detection and subsequent remediation. They also serve to assess the responsiveness and stability of monitored components, inform resource scaling decisions, and guide adjustments to scrape intervals. Ultimately, these indicators help to ensure the reliability of the entire observability pipeline within CROSS.

The performance metrics collected during evaluation offer important insights into system behaviour and optimisation priorities. Android exhibited the

highest number of errors, highlighting the need for further refinement of remediation strategies tailored to the platform's instability. Linux, on the other hand, recorded the greatest number of warnings, suggesting that more sophisticated predictive analytics or threshold adjustments will be required to avoid excessive alerts. Query latency remained minimal across all monitored components, confirming that real-time performance monitoring operates efficiently. Additionally, memory and CPU usage were stable throughout, demonstrating that resource management within the CROSS architecture is effective. Both scraping and configuration reloading processes performed optimally, ensuring sustained system responsiveness and minimal overhead in the telemetry pipeline.

Discussion

The CROSS framework addresses critical challenges in the design and deployment of self-healing systems, particularly within heterogeneous CPS environments. Unlike existing solutions that tend to focus on either detection or remediation within narrow operating boundaries, CROSS presents a holistic, platform-agnostic approach that fuses detection, remediation, and observability into a cohesive, modular architecture. It not only introduces a practical and extensible design for autonomous remediation but also raises the benchmark for cross-platform self-healing systems by combining fault detection, execution logic, and system introspection within a lightweight, scalable solution. While CROSS currently leverages lightweight probabilistic models for anomaly detection and employs threshold-based remediation, future enhancements may draw on advances in AIOps (Artificial Intelligence for IT Operations), as explored by Cheng et al. (2023). Their review highlights the growing role of machine learning in cloud-native diagnostics, root cause analysis, and intelligent automation; directions that align closely with CROSS's development roadmap, including reinforcement learning or predictive models to support more context-aware remediation decisions. This section critically evaluates the broader implications of the framework, distinguishing it from existing solutions and exploring its potential real-world applications.

CROSS demonstrates considerable adaptability through its successful deployment of remediation logic across Android, Linux, macOS, and Windows environments. This flexibility is achieved through the use of a dynamic registry for OS-specific handlers, which supports extensibility by allowing additional platforms to be incorporated without restructuring the core logic. In this regard, CROSS outperforms other solutions that are typically bound to a single operating system or dependent on rigid, static rule sets. Implementation testing revealed meaningful insights and challenges. For example, the

system effectively managed variations in available tooling and operating constraints across platforms, particularly those arising from containerisation, such as process ID (PID) isolation and limited access to system toolchains. These operational conditions underscored the importance of robust fallback routines and precise error handling. CROSS addresses this by simulating executions where necessary and maintaining comprehensive logging, thereby ensuring that remediation attempts are traceable and continue even under less-than-ideal conditions.

In terms of strategic positioning, CROSS aligns with emerging industry trends favouring autonomous, AI-driven infrastructure. Its integration with standard observability stacks, such as Prometheus and Grafana, supports low-latency recovery loops and quantifiable remediation metrics. Its lightweight classification approach and threshold-triggered execution logic make it particularly well-suited to resource-constrained environments, such as embedded or edge CPS deployments. Nonetheless, the framework has its limitations. Chief among these is the inability to invoke certain native remediation tools within containerised environments, an issue stemming from restricted process control and privilege boundaries. There may also be minor delays introduced by log processing or orchestration overhead. These limitations are offset to a degree by the system's modular design, transparent logging architecture, and OS-aware remediation logic. Future work may include the development of native agents or elevated runtime modules to bridge these gaps more effectively.

Adaptive fallback logic and CPS-relevant scenarios

Adaptive fallback logic provides resilience in situations where standard remediation routines cannot be executed due to unavailable tooling, permission restrictions, or degraded system states. Each remediation action is associated with one or more fallback alternatives, organised in a prioritised sequence. If an action fails, for example, due to missing binaries or command rejection, the fallback is automatically attempted, enabling CROSS to maintain operational continuity. In CPS contexts, fallback logic is particularly valuable.

For example, if a soft reset of a network service on an industrial gateway fails, CROSS may revert to a bounded interface reset, followed by a policy-defined escalation such as transferring the affected node into a maintenance-safe state. Similarly, should an edge controller fail to terminate an unresponsive process, CROSS may execute a container-level restart or, in highly constrained real-time environments, trigger a supervisory watchdog rather than performing system-wide restarts. These examples demonstrate that the fallback logic extends beyond conventional IT-centric

remediation, enabling CPS-specific recovery behaviours such as industrial fail-safe mechanisms, Remote Terminal Unit (RTU) state reinitialisation, and Programmable Logic Controller (PLC)-supervised inhibition states.

Policy-driven remediation

CROSS employs a policy-driven remediation layer that systematically maps detected anomalies to platform-specific corrective actions. Policies are specified declaratively using a lightweight rules format, allowing operators to define conditions, triggers, and associated remediation procedures without modifying source code. A generic policy entry consists of: (i) an anomaly signature or label; (ii) contextual constraints such as platform, severity, or resource thresholds; and (iii) a sequence of remediation actions with optional escalation logic. At runtime, CROSS matches predicted anomalies with the policy catalogue to determine an appropriate remediation plan.

Policies are evaluated in a context-aware manner: for example, a high-CPU anomaly on Linux may trigger process throttling, whereas an equivalent anomaly on Android may invoke app-level restarts due to platform constraints. Policy inheritance allows common remediation strategies to be specialised for individual OS families, ensuring consistency while enabling fine-grained platform tailoring. To illustrate, a network-degradation anomaly may trigger different sequences depending on execution context: (a) on a supervisory industrial gateway, CROSS may restart affected services or reset network interfaces; (b) on a safety-critical device, policies may restrict CROSS to non-intrusive diagnostics only, deferring intrusive actions to a supervisory controller. This separation ensures that remediation remains aligned with operational and safety requirements across heterogeneous CPS environments.

Interpreting resource impact in representative CPS hardware classes

The reported CPU and memory overheads must be interpreted in relation to the heterogeneous hardware found in CPS deployments. While CROSS was evaluated on general-purpose computing platforms, many CPS nodes, including PLCs, RTUs, and embedded protection devices-operate with significantly constrained resources and rely on real-time firmware rather than a full OS stack.

The observed overheads (typically under X% CPU and Y MB memory) are therefore most appropriate for CPS components with an IT-oriented architecture, such as industrial edge gateways, substation automation servers,

medical IoT hubs, or container-capable embedded Linux devices. Legacy PLCs and microcontroller-based systems are not target deployment points; instead, CROSS can function as a supervisory node augmenting such devices. In these configurations, resource impact is confined to the supervisory host, while CPS devices benefit indirectly through reduced mean-time-to-recovery (MTTR), improved anomaly visibility, and consistent remediation orchestration. This contextualisation clarifies that CROSS complements a layered CPS architecture: lightweight schemes such as LISH serve highly constrained environments, whereas CROSS delivers comprehensive remediation on more capable nodes within the CPS hierarchy.

CROSS exhibits significant potential for deployment in mission-critical domains such as healthcare monitoring, intelligent transport systems, smart grids, and industrial automation. Its observability model provides real-time insights into fault patterns, recovery latency, and overall system health. This, in turn, strengthens post-incident analysis, regulatory compliance, and operational accountability-features that are vital for environments where system reliability is non-negotiable. To deepen this perspective, the following subsection critically reflects on the limitations of using OS-level telemetry as an abstraction for CPS state, and how this impacts diagnosis and remediation in control-dominated environments.

A limitation of the present study is that CROSS was not evaluated directly on physical CPS or OT devices such as PLCs, RTUs, or protection relays. The experiments relied on OS-level telemetry as a unifying abstraction for demonstrating cross-platform anomaly detection and remediation. While this approach provides broad generalisability across heterogeneous operating systems, OS logs do not capture CPS-specific process deviations or control-layer faults observable only through field-bus traffic, PLC logic traces, or real-time controller telemetry. OS-level signals should therefore be viewed as a foundational but incomplete proxy for CPS behaviour. Future work will integrate cyber-physical system-native telemetry sources, including Distributed Network Protocol (DNP3) and Modbus communication events, Open Platform Communications Unified Architecture (OPC UA) process variables, and PLC execution traces, to enable deeper and more realistic evaluation in industrial environments.

Limitations of OS-level telemetry as a proxy for CPS insight

While OS-level telemetry provides a valuable and uniform abstraction for cross-platform anomaly detection, its suitability as a proxy for CPS-specific process anomalies has inherent limitations. In many operational technology (OT) domains, faults manifest not in the general-purpose operating system but within specialised

embedded subsystems. Examples include ladder-logic deviations in programmable logic controllers (PLCs), timing faults detectable only on field-bus networks (for example, Modbus, CAN, Profibus), or domain-specific measurements observed within real-time controllers and remote terminal units (RTUs).

Such anomalies may not propagate to the host operating system and therefore remain invisible to generic OS logs. CROSS addresses this by treating OS-level signals as a baseline source of behavioural deviation rather than a complete view of CPS health. This design enables broad deployability but does not replace the need for specialised telemetry in safety- or mission-critical environments. Future extensions could incorporate heterogeneous log sources such as PLC execution traces, DNP3 traffic summaries, or OPC-UA telemetry, enabling a multi-layered view that unifies OS and control-layer anomalies. This hybrid approach would enhance diagnostic richness, particularly for faults arising at the physical–digital boundary where OS-level signals alone are insufficient.

Deployment considerations for real-time, deterministic, and air-gapped CPS

Cyber-physical system deployments frequently operate under stringent constraints related to timing, determinism, and connectivity. Systems such as industrial protection relays, aerospace control units, and medical devices often enforce fixed execution cycles, bounded timing jitter, and, in some domains, strictly air-gapped operational modes. The cloud-native design of CROSS does not inherently require wide-area network connectivity; instead, its components can be deployed entirely on premises or within isolated edge clusters using lightweight, locally managed Kubernetes container orchestration distributions, such as K3s, a compact Kubernetes implementation designed for resource-constrained and edge environments, and MicroK8s, a self-contained Kubernetes platform optimised for local, offline, and air-gapped deployments. These configurations preserve equivalent functionality to cloud-connected deployments while remaining compatible with security-sensitive CPS settings.

Real-time scheduling constraints are accommodated by executing anomaly detection and remediation tasks as background processes with bounded resource footprints, thereby ensuring minimal interference with time-critical control workloads. Where strict real-time determinism is required, CROSS can be configured to execute remediation actions only within predefined safe execution windows or to defer intrusive actions to higher-level supervisory nodes. In fully air-gapped environments, all telemetry storage, policy management, and inference

routines operate locally, eliminating any dependency on external cloud services. Collectively, these deployment modes extend the applicability of CROSS to legacy, safety-critical, and infrastructure-constrained cyber-physical system environments.

Key Contributions. To clearly summarise its novelty and impact, CROSS delivers the following core contributions:

- *First cloud-native, cross-platform remediation framework:* Enables automated recovery across Android, Linux, macOS, and Windows, triggered via machine learning-based log classification.
- *Integrated observability with remediation:* Prometheus and Grafana integration allows real-time inspection of errors, recovery events, and healing metrics.
- *Simulated fallback execution:* Ensures continuity by simulating remediation steps when native tools (for example, ADB or PowerShell) are unavailable in restricted environments.
- *Modular and extensible architecture:* A registry-driven design allows easy addition of new platforms and remediation handlers without altering the core system.
- *Validated against baselines:* Demonstrates superior MTTR, precision, and recall compared to manual scripts and rule-based systems.

Conclusion and future work

Although CROSS integrates anomaly detection mechanisms that are applicable to cybersecurity contexts, the present evaluation focuses primarily on operational anomalies rather than explicit attack-induced behaviours. As such, the cybersecurity contribution is framed as an architectural capability rather than an empirically benchmarked claim. Evaluating CROSS using intrusion-specific datasets is identified as an important direction for future work.

This study introduced CROSS, the Cross-platform Remediation and Observability Self-Healing System, as a novel framework for autonomous fault recovery in cyber-physical systems. Through lightweight log-based anomaly detection, containerised deployment using Docker, scalable orchestration via Kubernetes, and an observability layer powered by Prometheus and Grafana, CROSS achieves a balanced combination of adaptability, transparency, and performance. Unlike many existing self-healing solutions confined to specific platforms or

toolchains, CROSS dynamically adjusts its remediation logic to suit Android, Linux, macOS, and Windows environments, maintaining operational continuity even in constrained or hybrid execution contexts.

Beyond its practical feasibility, CROSS makes a broader research contribution by bridging theoretical models of fault tolerance with deployable, real-world self-healing mechanisms. It executes corrective actions autonomously while emitting verifiable telemetry, thereby strengthening both system resilience and auditability. The framework demonstrates that explainable log-driven learning can underpin secure, cross-platform self-healing architectures suitable for industrial CPS domains such as smart grids, healthcare monitoring, and intelligent transport.

Integrated roadmap for future work

Future research will extend CROSS along several complementary directions:

- *Adversarial Robustness and Security Context:* Incorporate adversarial training and ensemble detection to resist evasion and poisoning attacks targeting log-based classifiers, enabling the framework to distinguish operational faults from malicious anomalies.
- *Reinforcement-Learning-Driven Remediation:* Model remediation as a Markov Decision Process (MDP) where adaptive policies learn optimal corrective actions from feedback, balancing availability with security risk.
- *Predictive and Time-Series Analytics:* Employ supervised and temporal learning models for proactive fault prediction and pre-emptive remediation before service degradation occurs.
- *Secure and Auditable Execution:* Introduce sandboxed or privilege-restricted healing agents and explore blockchain-based audit trails to enhance the trustworthiness and verifiability of remediation actions.
- *Edge-Cloud Hybrid Architecture:* Distribute detection and remediation across computing tiers, latency-sensitive operations at the edge and orchestration in the cloud, to support large-scale CPS deployments.
- *Human-in-the-Loop Interfaces:* Develop operator dashboards that allow manual override, validation, and contextual decision support, blending autonomy with expert insight.

These directions aim to evolve CROSS from an operationally self-healing engine into a fully cyber-resilient, explainable, and trustworthy ecosystem for heterogeneous CPS. The framework thus establishes a foundation for scalable, intelligent, and transparent self-healing infrastructures across both IT and OT domains.

Acknowledgements

The authors would like to acknowledge the support provided by the Cyber Security Research Group (CSRG), Department of Computer Science, Nottingham Trent University.

Author contributions

Conceptualisation: O.J. and A.S.S.; Methodology: O.J.; Software: O.J.; Validation: O.J., A.S.S., and O.K.; Analysis: O.J. and A.S.S.; Resources: A.S.S., O.K., and M.A.; Data curation: O.J.; Writing - original draft: O.J.; Writing - review and editing: O.J., A.S.S., O.K., and M.A.; Visualisation: O.J.; Supervision: A.S.S. and O.K. All authors read and approved the final manuscript.

Funding

This research received no external funding.

Data availability

The source code of the CROSS framework is openly available on GitHub ([30]): <https://github.com/obinnajohnphill/self-healing-remedial-actions> (accessed on 19 April 2025). The preprocessed dataset, including raw logs, extracted features, and processed data derived from the publicly available Loghub repository curated by LOGPAI ([26, 27]), is available on Zenodo ([28]): <https://doi.org/10.5281/zenodo.15249598> (accessed on 19 April 2025).

Declarations

Ethics approval

Not applicable.

Consent for publication

Not applicable.

Competing interests

The authors declare that they have no Competing interests.

Received: 6 November 2025 Accepted: 11 January 2026

Published online: 22 February 2026

References

- Akinbolaji TJ, Nzeako G, Akokodaripon D, Aderoju AV (2024) Proactive monitoring and security in cloud infrastructure: leveraging tools like prometheus, grafana, and hashicorp vault. *World J Adv Eng Technol Sci* 13(2):74–89
- Ali T, Iqbal R, Ansari NM, Tariq T, Rafique AA (2025) Ai-powered anomaly detection in software logs: a machine learning approach for proactive fault diagnosis and self-healing systems. *Spectr Eng Sci* 3(3):302–322
- Arefifar SA, Alam MS, Hamadi A (2023) A review on self-healing in modern power distribution systems. *J Mod Power Syst Clean Energy* 11(6):1719–1733
- Cheng Y, Zhang W, Shen H, Tang C (2023) AI for IT operations (AIOps) on cloud platforms: reviews, opportunities and challenges
- Cloud native computing foundation: CNCF. Accessed 2 Jan 2025 (2025). <https://www.cncf.io/>
- Georgiou MA, Paphitis A, Sirivianos M, Herodotou H (2019) Towards auto-scaling existing transactional databases with strong consistency. In: *Proc. IEEE 35th ICDE Workshops (ICDEW)*, pp. 107–112
- Hallur J (2022) From monitoring to observability: enhancing system reliability and team productivity. *Int J Sci Res (IJSR)* 13(10):602–606
- He P, Zhu J, Zheng S, Li Z, Lyu MR (2023) Loghub: a large collection of system log datasets for ai-driven log analytics. In: *Proc. IEEE ISSRE*, pp. 1–11 IEEE. <https://github.com/logpai/loghub>
- Jhuang C, Wu Q (2022) Container-based voip failover design on kubernetes overlay networks. In: *Proc. CSCS*, pp. 1317–1322
- Johnphill O, Sadiq AS, Kaiwartya O (2024) Preprocessed log dataset for cross-platform remedial actions in cyber-physical systems. <https://doi.org/10.5281/zenodo.15249598>
- Johnphill O, Sadiq AS, Kaiwartya O, Aljaidi M (2024) An intelligent approach to automated operating systems log analysis for enhanced security. *Information* 15:657

- Johnphill O, Sadiq AS, Pillai P, Taheir MA, Kaiwartya O (2023) Self-healing in cyber-physical systems using machine learning: a critical analysis of theories and tools. *Futur Internet* 15(7):244
- LOGPAI: Loghub: a large collection of system log datasets for AI-driven log analytics (2023). <https://github.com/logpai/loghub>
- Loh R, Thing VLL (2023) Enhancing cyber-resilience in self-healing cyber-physical systems with implicit guarantees. In: *Proc. IEEE International Conference on Cyber Security and Resilience (CSR)*, pp. 359–366
- Malhotra A, Elsayed A, Torres R, Venkatraman S (2023) Evaluate solutions for achieving high availability or near zero downtime for cloud native enterprise applications. *IEEE Access* 11:85384–85394
- Marinho CSS, Moreira LO, Coutinho EF, Costa Filho JS, Sousa FRC, Machado JC (2018) Labareda: a predictive and elastic load balancing service for cloud-replicated databases. *J Inf Data Manag* 9(1):94–106
- Naqvi S, Ahmed T, Ullah M (2022) On evaluating self-adaptive and self-healing systems using chaos engineering. In: *Proc. IEEE international conference on autonomic computing and self-organizing systems (ACSOS)*, pp. 1–10
- Nugroho MA, Abdurrohman M, Erfianto B (2023) Increasing automatic proctoring system performance using distributed round-robin load balancer. In: *Proc. IEEE IAICT*, pp. 377–382
- Pai K, Srinivas BK (2024) Enhanced visibility for real-time monitoring and alerting in kubernetes by integrating prometheus, grafana, loki, and alerta. *Int J Sci Res Eng Manag IJSREM* 8(6):1–5
- Pakrijauskas K, Mazeika D (2022) Investigation of stateful microservice availability during failover. In: *Proc. 8th International Conference on Control, Decision and Information Technologies (CoDIT)*, pp. 286–290
- Shabariram CP, Vrinda S, Srivastan S, Manimeghalai R (2024) Case study based investigation on self-healing cloud deployments for edge-based software development. In: *Proc. ICSSECC*, pp. 85–90
- Siddiqui I, Pandey A, Jain S, Kothadia H, Agrawal R, Chankhore N (2023) Comprehensive monitoring and observability with jenkins and grafana. In: *Proc. ISMSIT*, pp. 1–5
- Vervaeke A (2021) Monilog: An automated log-based anomaly detection system for cloud computing infrastructures. In: *Proc. IEEE 37th international conference on data engineering (ICDE)*, pp. 2739–2743
- Wang Y, Xu Y, He J, Liu C, Schneider KP, Hong M, Ton DT (2019) Coordinating multiple sources for service restoration to enhance resilience of distribution systems. *IEEE Trans Smart Grid* 10(5):5781–5793

Publisher's Note

Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.